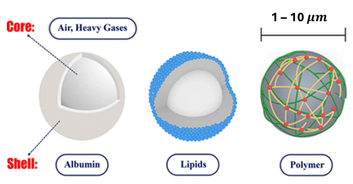
# FPGA Implementation of Chirp Coded Subharmonic Ultrasound Imaging Using Nonlinear Volterra Filtering

Amartya Tuljapurkar (24110034), Divisht (24110112), Helly Bera (24110133), Aryan, and Ruchika Dhawan under the supervision of Prof. Joycee Mekie

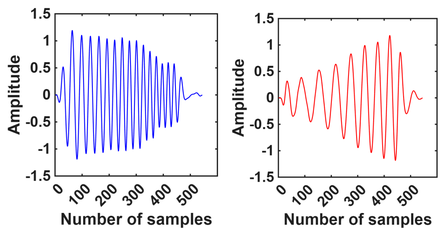
Department of Electrical Engineering, Indian Institute of Technology Gandhinagar

## Background and Motivation

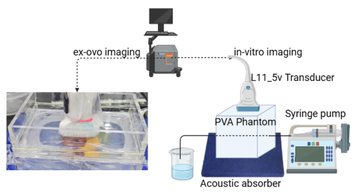
- Microbubbles are gas-filled contrast agents that exhibit nonlinear oscillations, unlike surrounding tissue.
- They produce subharmonic ($f_0/2$) and ultraharmonic components, enabling selective detection and improved imaging contrast.




a) Microbubble

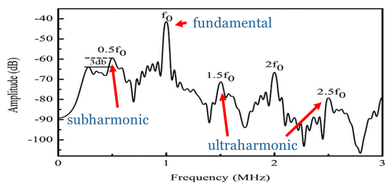



b) Fundamental chirp (blue) and subharmonic (red) waveforms

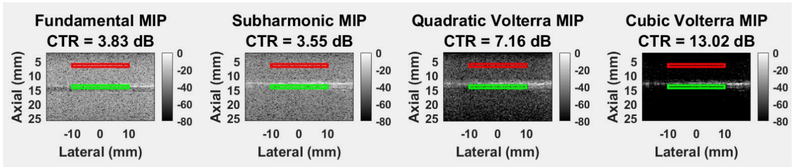



c) Experimental Setup

- Chirp-coded excitation increases signal duration (not amplitude), enhancing energy transmission and improving detection of nonlinear responses.
- Extracting these components requires intensive processing, motivating the implementation of a multi-stage pipeline on FPGA.



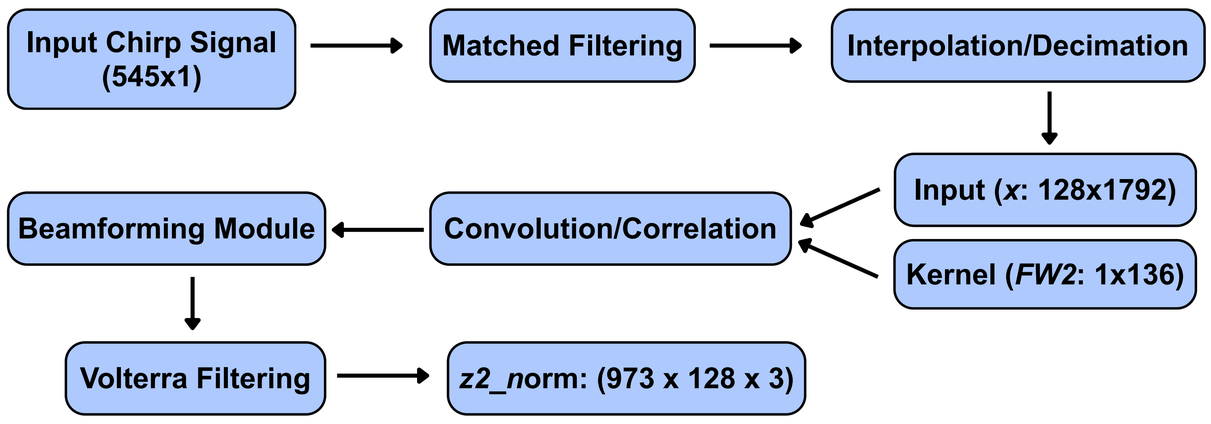

d) Frequency response




e) MATLAB Output contrast comparison

## Pipeline

- The system processes ultrasound data of size 1792 x 128 x 100, where 1792 corresponds to time samples, 128 to transducer elements, and 100 to frames acquired sequentially.
- The pipeline performs interpolation, matched filtering(convolution), and beamforming to align and enhance the received signals, followed by Volterra filtering for nonlinear processing.




$$y[n] = \sum_{i=0}^{m-1} x[n-i]h_l[i] + \sum_{i=0}^{m-1}\sum_{j=0}^{m-1} x[n-i]x[n-j]h_q[i,j] + \sum_{i=0}^{m-1}\sum_{j=0}^{m-1}\sum_{k=0}^{m-1} x[n-i]x[n-j]x[n-k]h_c[i,j,k] = y_l[n] + y_q[n] + y_c[n]$$

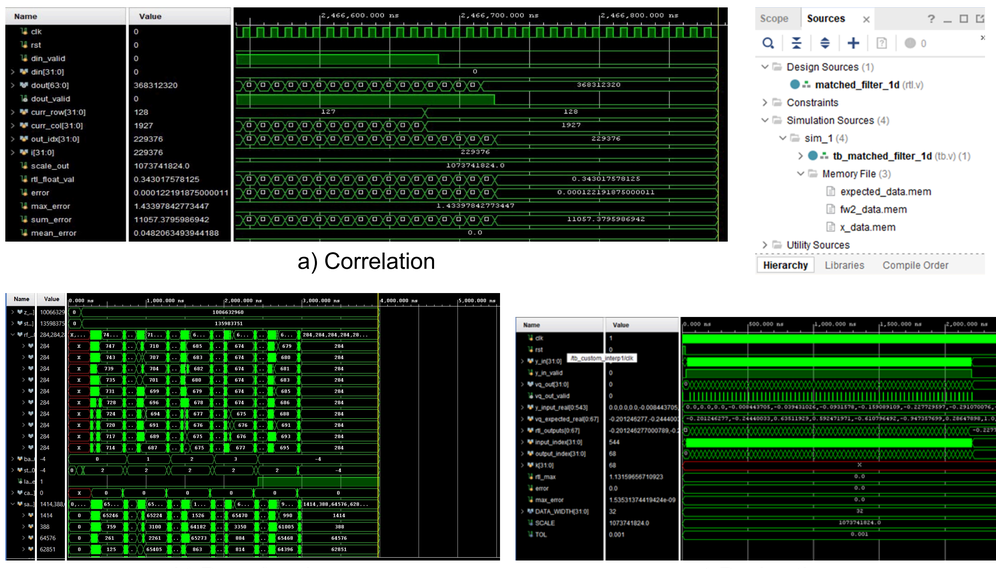
Nonlinear Volterra Filtering Model

- The final output is a dataset of size 973 x 128 x 3, representing reduced and refined signal components after multi-stage processing.

## Methodology

- All modules were implemented in Verilog HDL, verified using phantom input data in Vivado, and validated against MATLAB outputs.
- Fixed-point arithmetic was used to replace floating-point operations, enabling efficient hardware implementation while maintaining high numerical accuracy.
- The design was optimized using DSP slices, pipelining, and parallelism, and successfully synthesized on the Nexys4 DDR board as individual modules.

## Simulation Waveforms

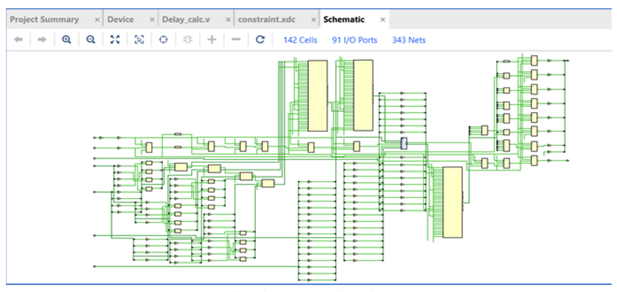


a) Correlation

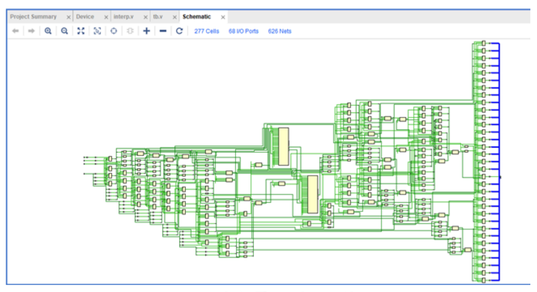
b) Beamforming

c) Decimation

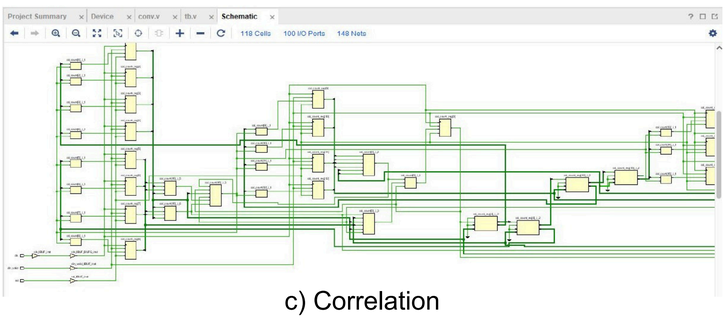
## Synthesis Schematics



a) Delay Calculator



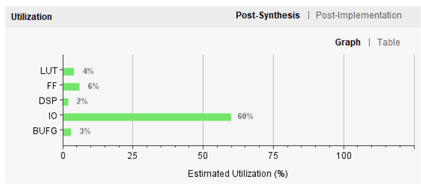
b) Decimation



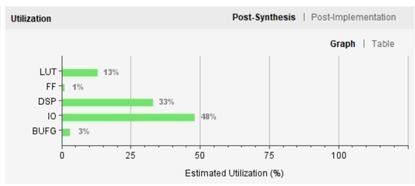
c) Correlation

## Utilization Reports

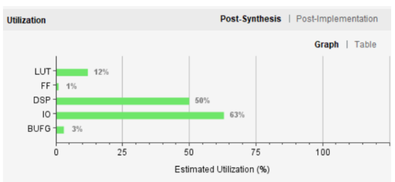
- Resource utilization increases with computational complexity, with quadratic and cubic product modules (which are non-linear) showing higher DSP usage due to intensive multiplication operations.



a) Dot product



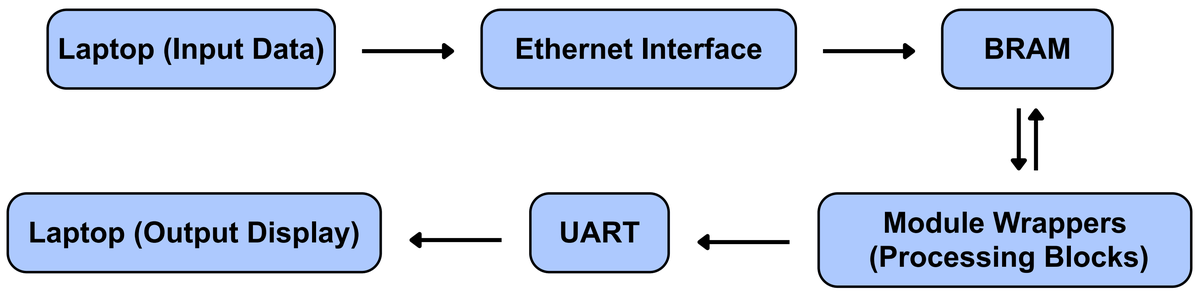
b) Quadratic product



c) Cubic product

## Data Transfer Method

- Input data is transferred from an external device (laptop) to the FPGA using an Ethernet interface, allowing high-speed data communication.
- The received data is stored in an on-chip BRAM, which acts as a temporary storage for easier access by the processing modules.




- Module wrappers read input data from BRAM, perform the required computations, and generate outputs.
- The computed results are transmitted back to the external device via UART, completing the end-to-end data flow pipeline.

## Results

- All modules demonstrate high numerical accuracy when compared with MATLAB outputs, with most errors in the range of $10^{-5}$ to $10^{-8}$, validating the correctness of the hardware implementation.
- Successful synthesis of all modules confirms that the designs are hardware-efficient and ready for integration into a complete FPGA-based pipeline.

| Module | Metric | Error |
|---|---|---|
| Interpolation | RMSE | ~$10^{-8}$ |
| Convolution | RMSE | $2.02 \times 10^{-3}$ |
| Dot Product | Absolute Error | $1.45 \times 10^{-8}$ |
| Volterra (2nd Order) | RMSE | $2.88 \times 10^{-5}$ |
| Volterra (3rd Order) | RMSE | $9.67 \times 10^{-7}$ |
| Square Root (CORDIC) | RMSE | 0.059% |
| Matrix Flip | Max Error | $3.1 \times 10^{-5}$ |

## Summary and Future Work

- Integration of all the independent modules into a complete pipeline.
- Hardware deployment and testing on larger FPGA boards to check performance with real data.
- Optimization of the developed pipeline to reduce latency, improve resource utilization, and reduce overall errors.
- SVD implementation after the beamforming function for enhanced signal analysis.